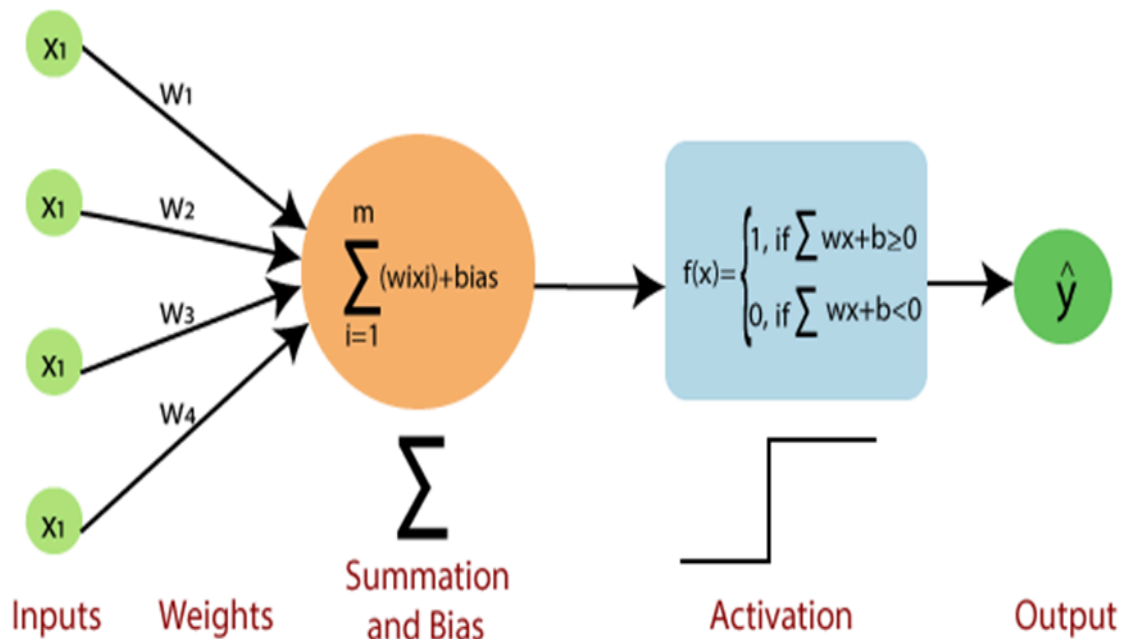


Perceptrons & Activation Functions: Foundations of Neural Networks

Overview

The **Perceptron** is the building block of a neural network. It mimics a biological neuron:

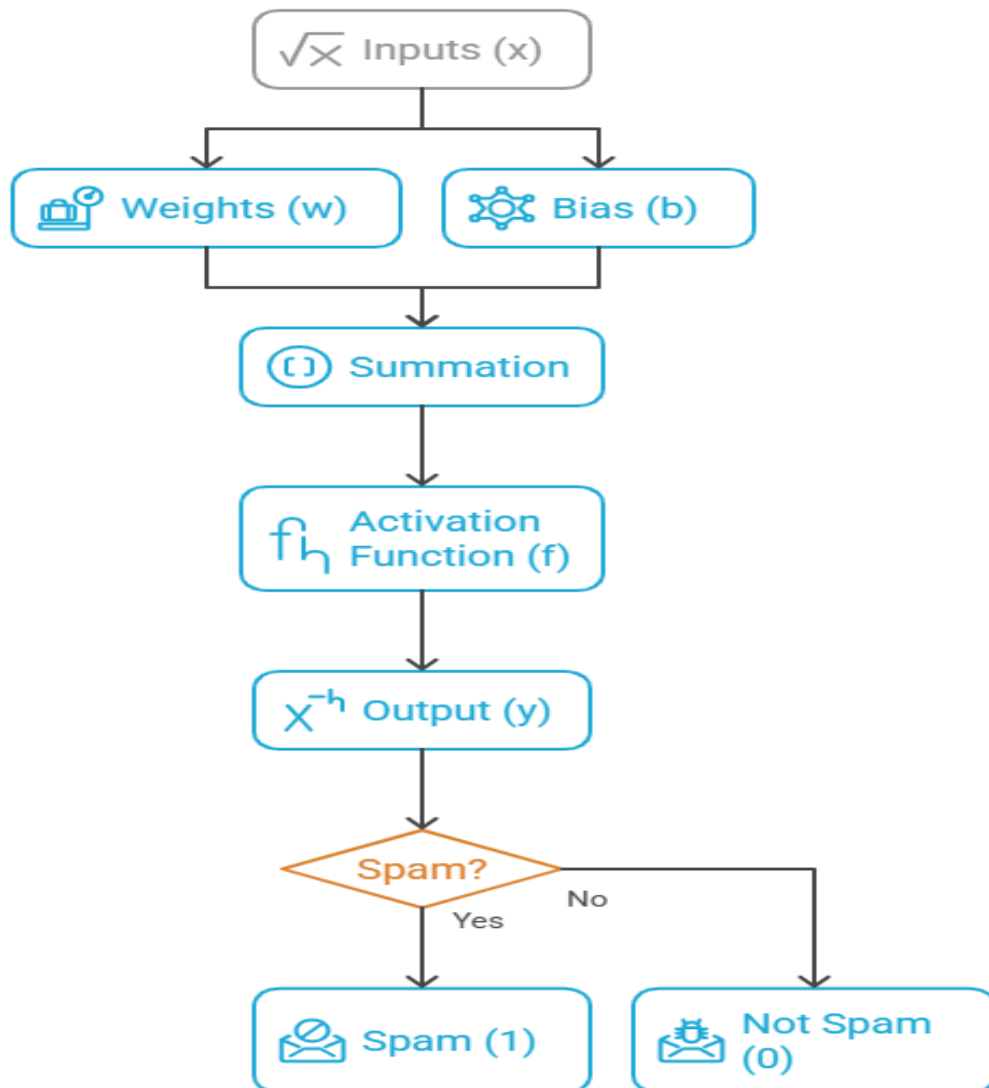
- Inputs come in (like signals to dendrites).
- Weights control the importance of each input.
- A bias shifts the decision boundary.
- An **activation function** decides the output.



Without activation functions, neural networks would just be **linear models** — they wouldn't capture complex patterns.

Perceptron Model

Perceptron Model Flowchart



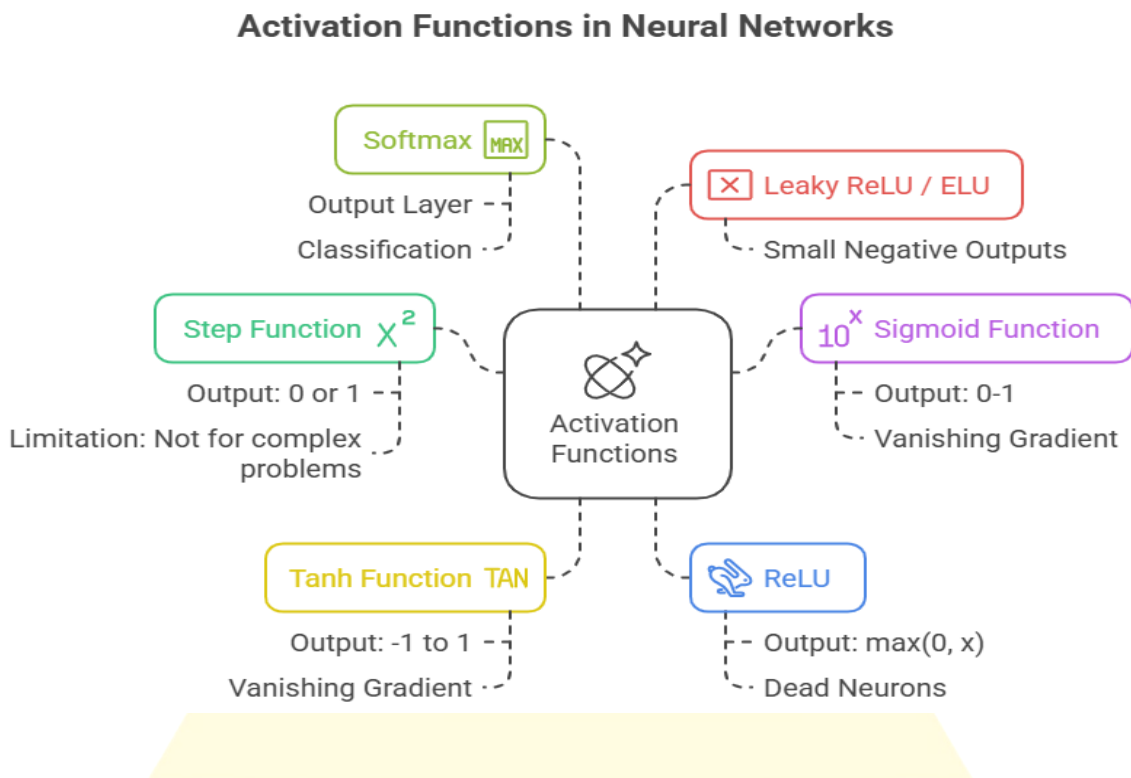
Equation:

$$y = f(\sum w_i x_i + b)$$

- **Inputs (x):** Features of data (e.g., pixels of an image).
- **Weights (w):** Learnable parameters (importance of each input).

- **Bias (b):** Adjusts decision threshold.
- **Activation (f):** Non-linear function that decides the output.
Example: Spam email filter
- Inputs = features like "word frequency," "contains links," etc.
- Perceptron learns weights to decide: Spam (1) or Not Spam (0).

Activation Functions



1. Step Function

- Output is 0 or 1.
- Used in the first perceptrons.
- **Limitation: Not useful for complex problems.**

2. Sigmoid Function

$$f(x) = \frac{1}{1 + e^{-x}}$$

- Smooth, outputs between 0–1.
- Good for probabilities.
- Vanishing gradient problem.

3. Tanh Function

- Output between -1 and 1.
- Centered around 0 (better than sigmoid).
- Still suffers from vanishing gradients.

4. ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

- Fast and simple.
- Most widely used in DL.
- Dead neurons (if too many outputs = 0).

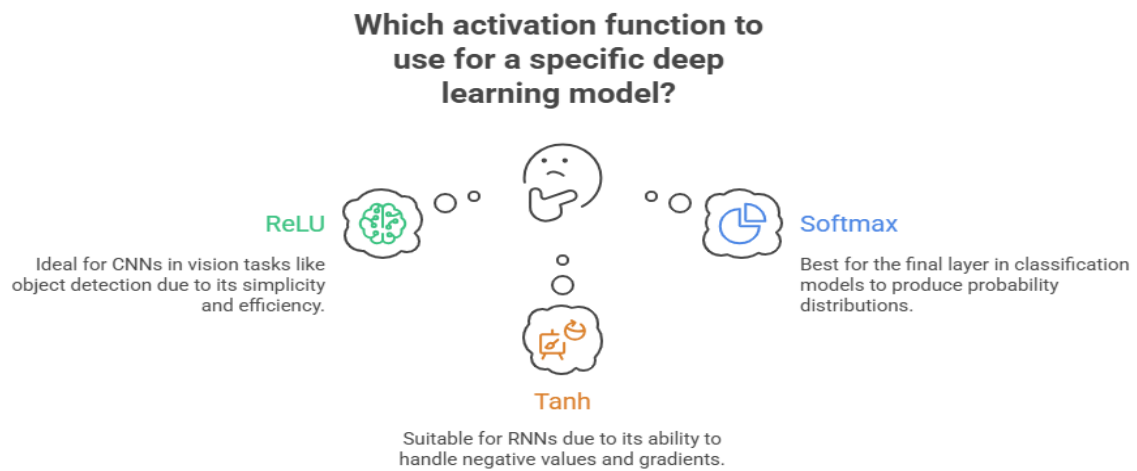
5. Leaky ReLU / ELU

- Fix dead neuron issues by allowing small negative outputs.

6. Softmax

- Converts scores into probability distribution.
- Used in the **output layer** for classification.

DL Applications



- ReLU → CNNs for vision (e.g., object detection).
- Softmax → Final layer in classification models.
- Tanh → Used in RNNs.

INGRADE